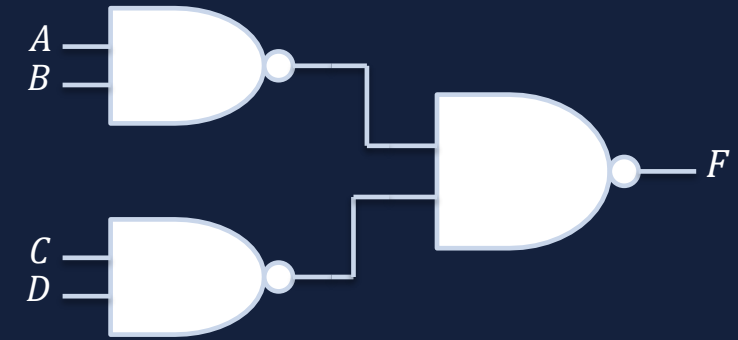


Lecture 7

NAND, NOR and XOR Implementations

NAND and NOR are universal · bubble pushing · multilevel circuits · AOI and OAI · XOR and parity · gate-level Verilog

Reading: Mano & Ciletti, *Digital Design with an Introduction to the Verilog HDL*, 6th ed. — Chapter 3, §3.6–3.9



$$F = AB + CD$$

Today's roadmap

1

§3.6

NAND circuits

Universal gate; bubble pushing

2

§3.6

NOR circuits

The dual, for POS

3

§3.7

Other forms

AOI, OAI, wired logic

4

§3.8

XOR and parity

Odd functions; Verilog

By the end of class you can...

- Build NOT, AND and OR from NAND or NOR alone
- Convert AND-OR to NAND-NAND and OR-AND to NOR-NOR
- Convert multilevel circuits by pushing bubbles
- Design parity generators and checkers with XOR
- Describe a gate circuit with delays in Verilog

Agenda for today's session

Intro	NAND	Break	NOR	Forms	XOR	Wrap	
0:00	0:05	0:35	0:45	1:00	1:10	1:35	1:50

Elapsed time	Topic	In-class work
0:00 – 0:05	Welcome, roadmap	Homework 3 collected
0:05 – 0:35	Universal NAND, symbols, bubble pushing	Examples 1, 2 · Your turn 1
0:35 – 0:45	Break	
0:45 – 1:00	NOR-NOR from product of sums	Example 3
1:00 – 1:10	AOI, OAI and wired logic	—
1:10 – 1:35	Exclusive-OR, odd functions, parity	Example 4 · Your turn 2
1:35 – 1:50	Gate-level Verilog; wrap-up	Lab C preview

NAND is a universal gate

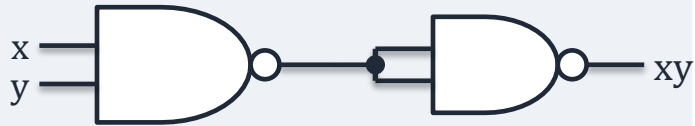
§3.6

NOT



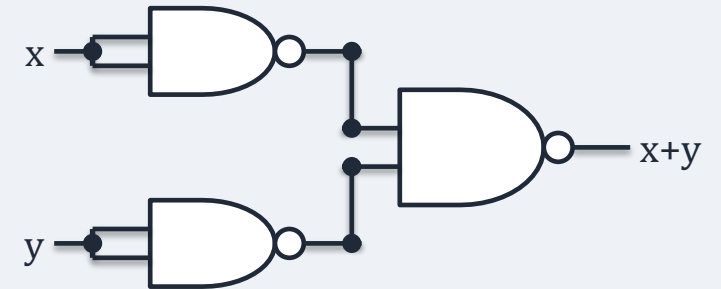
$$(xx)' = x'$$

AND



$$((xy)')' = xy$$

OR



$$(x'y')' = x + y$$

NOT, AND and OR together can build any function, so NAND alone can too.

In CMOS a NAND needs 4 transistors and an AND needs 6: AND is a NAND plus an inverter.

Two symbols for one gate

§3.6

NAND



AND-invert = invert-OR

$$(xyz)' = x' + y' + z'$$

NOR



OR-invert = invert-AND

$$(x + y + z)' = x'y'z'$$

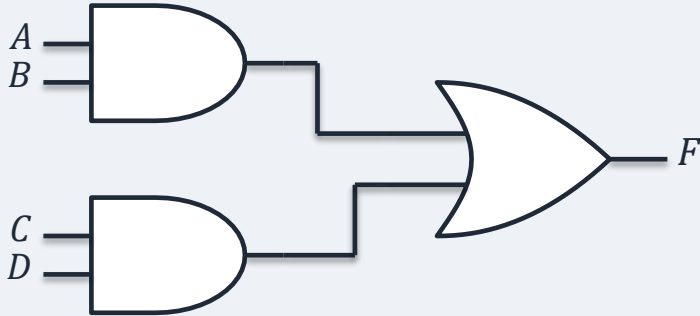
A bubble is a complement

Two bubbles on the same wire cancel: $(x')' = x$. Draw whichever symbol makes the bubbles meet, and the diagram reads like the original AND-OR or OR-AND circuit.

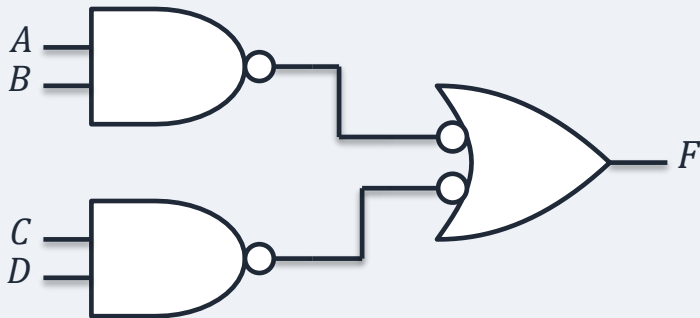
Procedure: AND-OR to NAND-NAND

§3.6

Start: AND-OR for $F = AB + CD$



Bubbles added: NAND-NAND



- 1 Simplify to SOP; draw AND gates feeding one OR.
- 2 Bubble every AND **output**: AND-invert NAND.
- 3 Bubble every OR **input**: invert-OR NAND.
- 4 Every bubble must face a bubble. A lone bubble on a literal: feed the complement or add an inverter.
- 5 Check: $[(AB)'(CD)']' = AB + CD$ ✓

Example 1 — Two-level NAND

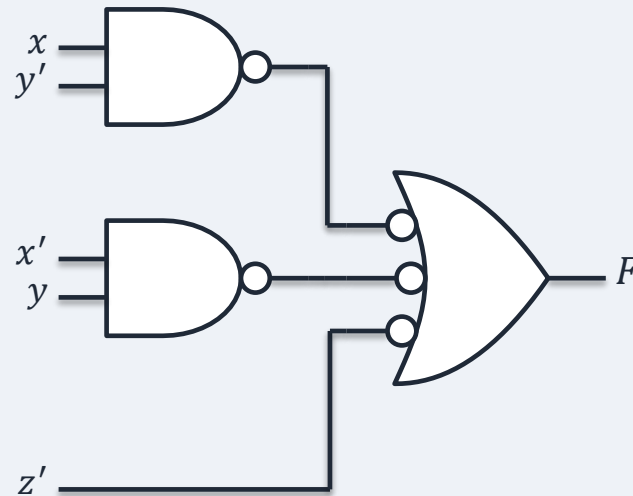
§3.6

EXAMPLE

Implement $F(x, y, z) = \Sigma m(1, 2, 3, 4, 5, 7)$ with NAND gates only.

		yz			
x		00	01	11	10
	0		1	1	1
	1	1	1	1	

NAND-NAND



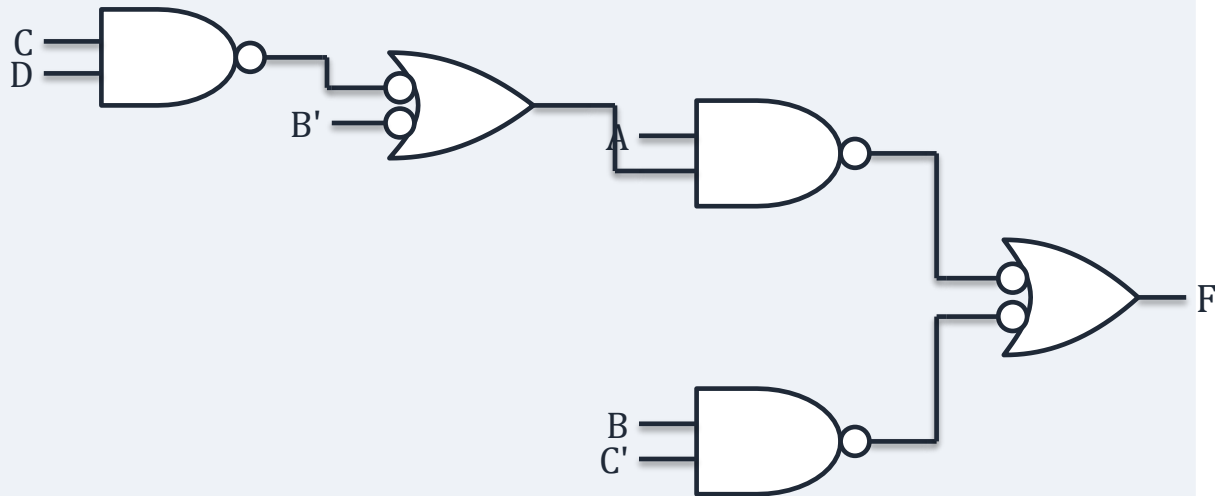
- 1 Map: quad m1, m3, m5, m7 ($z = 1$) $\rightarrow z$
- 2 Pairs m4, m5 $\rightarrow xy'$; m2, m3 $\rightarrow x'y$
- 3 SOP: $F = xy' + x'y + z$
- 4 Each AND $\rightarrow$ NAND; the OR $\rightarrow$ invert-OR (a NAND)
- 5 xy' , $x'y$ wires: bubble meets bubble $\checkmark$
- 6 Lone literal z meets an input bubble: unmatched
- 7 So feed z' there (or add an inverter)
- 8 Check: $[(xy')'(x'y)'(z')']' = xy' + x'y + z \checkmark$

Example 2 — Multilevel NAND

§3.6

EXAMPLE

Convert $F = A(CD + B) + BC'$ to NAND gates only.



- 1 Start from the AND-OR circuit as written.
- 2 Every AND $\rightarrow$ AND-invert (NAND, output bubble)
- 3 Every OR $\rightarrow$ invert-OR (NAND, input bubbles)
- 4 Internal wires: each output bubble meets an input bubble ✓
- 5 B enters a bubbled OR input directly: unmatched
- 6 So feed B' there
- 7 Check (DeMorgan): $[(A(CD + B))'(BC')']' = A(CD + B) + BC'$ ✓

Your turn 1 — NAND conversion

§3.6

YOUR TURN

≈ 10 min

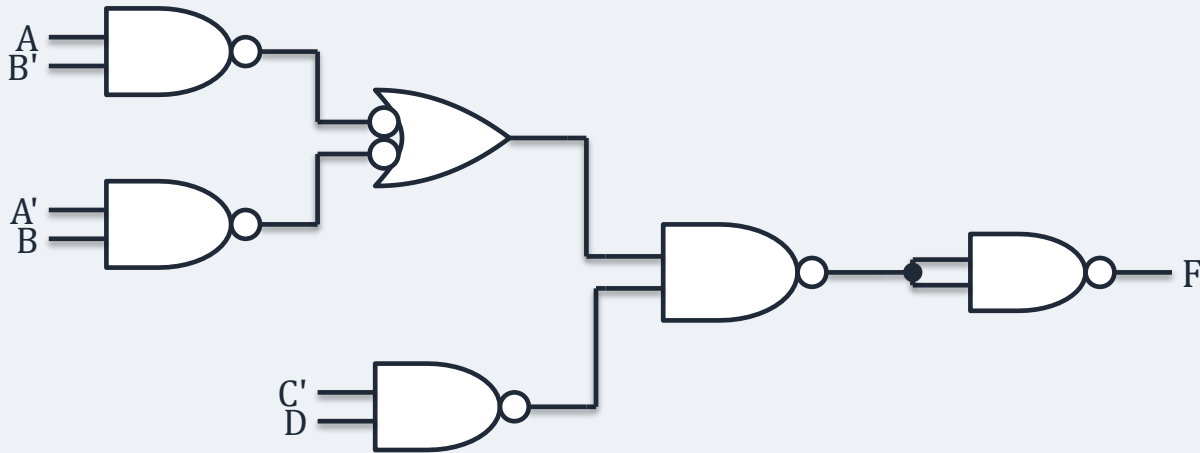
- (a) Convert $F = (AB' + A'B)(C + D')$ to NAND gates only. Watch the final AND gate.
- (b) A circuit has NAND gates on (x', y') and on (x, y) , an inverter on z , and a 3-input NAND combining all three. What is F ?

Count the bubbles on every path, including the output.

Your turn 1 — Solution

§3.6

SOLUTION



The final AND became a NAND with an unmatched output bubble, so a NAND wired as an inverter restores F .

- 1 (a) $AB' + A'B$: NAND-NAND, bubbles match
- 2 $C + D'$ is an OR $\rightarrow$ invert-OR: feed C' and D
- 3 Final AND $\rightarrow$ NAND: output bubble unmatched
- 4 Add an inverter (1-input NAND) to get F
- 5 (b) Gate outputs: $(x'y')'$, $(xy)'$ and z'
- 6 $F = [(x'y')'(xy)'z']'$
- 7 DeMorgan: $F = x'y' + xy + z$
- 8 $x'y' + xy = (x \oplus y)'$, so $F = (x \oplus y)' + z$

Break

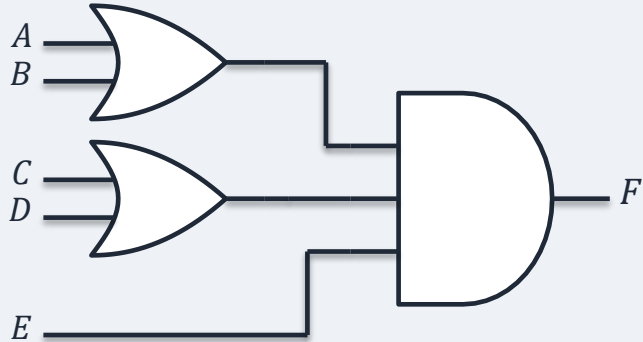
10 minutes — we resume at the 0:45 mark

Next: NOR-NOR circuits and the exclusive-OR

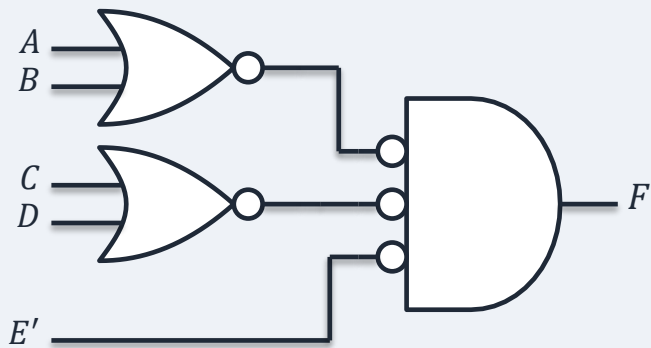
NOR is universal too: NOR-NOR

§3.6

OR-AND for $F = (A + B)(C + D)E$



NOR-NOR: note E'



The dual of NAND

NOT: $(x + x)' = x'$

OR: $((x + y)')' = x + y$

AND: $(x' + y')' = xy$

Procedure

Simplify to POS. Bubble every OR output and every AND input. Complement a literal that meets a lone bubble.

Check: $[(A + B)' + (C + D)' + E']' = (A + B)(C + D)E$

Example 3 — NOR-NOR from the map

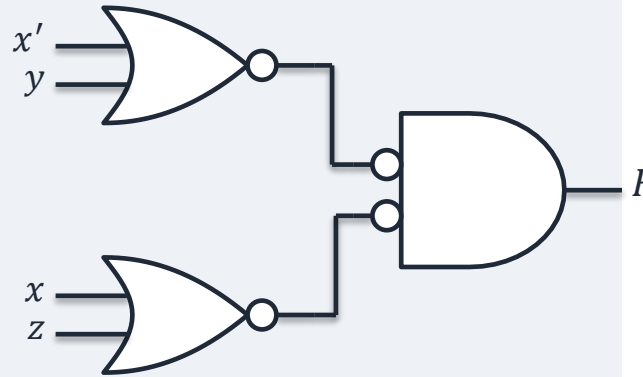
§3.6

EXAMPLE

Implement $F(x, y, z) = \Sigma m(1, 3, 6, 7)$ with NOR gates only.

x \ yz	00	01	11	10
0	0	1	1	0
1	0	0	1	1

NOR-NOR



- 1 Write 0s on the map: m0, m2, m4, m5
- 2 Group the 0s: m0, m2 $\rightarrow x'z'$; m4, m5 $\rightarrow xy'$
- 3 So $F' = x'z' + xy'$
- 4 DeMorgan: $F = (x + z)(x' + y)$
- 5 Each OR $\rightarrow$ NOR (output bubble)
- 6 The AND $\rightarrow$ invert-AND (a NOR): bubbles match
- 7 Check: $[(x' + y)' + (x + z)']' = xy + x'z \checkmark$

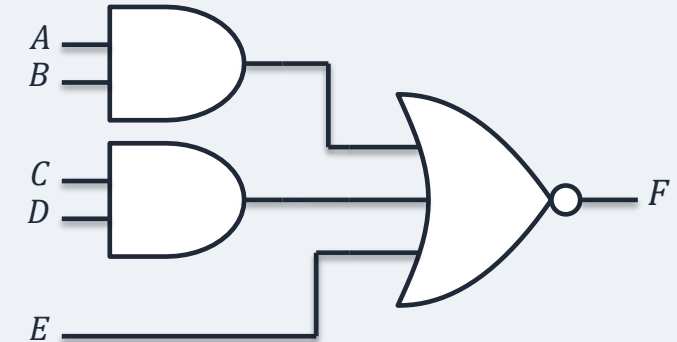
Other two-level forms

§3.7

Form	Equivalent	Function
AND-OR	NAND-NAND	SOP
OR-AND	NOR-NOR	POS
AND-NOR	NAND-AND	AND-OR-INVERT
OR-NAND	NOR-OR	OR-AND-INVERT
NOR-OR	—	degenerate (AND)
NAND-AND	—	degenerate (NAND)

Degenerate forms collapse to a single operation; the rest implement any function.

AND-OR-INVERT: $F = (AB + CD + E)'$



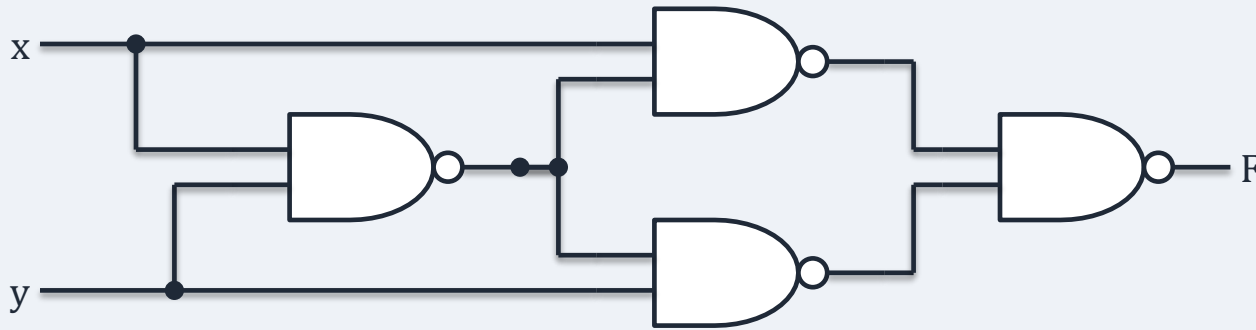
Wired logic

Open-collector outputs tied together with a pull-up form a wired-AND with no extra gate.

The exclusive-OR

§3.8

XOR from four NAND gates



$$F = [(x(xy)')'(y(xy)'))']' = x \oplus y$$

Four NANDs: why an XOR costs more than an AND or OR.

Identities

$$x \oplus y = xy' + x'y$$

$$x \oplus 0 = x, x \oplus 1 = x'$$

$$x \oplus x = 0, x \oplus x' = 1$$

$$x \oplus y' = x' \oplus y = (x \oplus y)'$$

commutative and associative

XNOR (equivalence)

$(x \oplus y)' = xy + x'y'$: 1 when the inputs are equal.

Odd and even functions

§3.8

$x \oplus y \oplus z$: odd function

		yz			
		00	01	11	10
x	0	m0	m1 1	m3	m2 1
	1	m4 1	m5	m7 1	m6

Odd function

1 when an odd number of inputs is 1: $\Sigma m(1,2,4,7)$ for three variables.

Even function

Its complement: $(x \oplus y \oplus z)' = \Sigma m(0,3,5,6)$.

Checkerboard maps

No two 1s are adjacent, so the SOP cannot shrink: 4 terms of 3 literals. Two XOR gates do it instead.

When you see a checkerboard, think XOR.

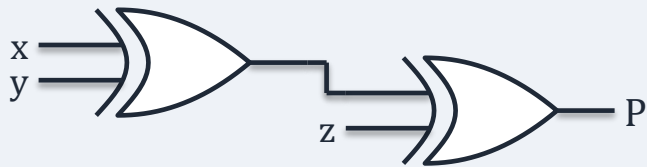
Example 4 — Parity: generate, check

§3.8

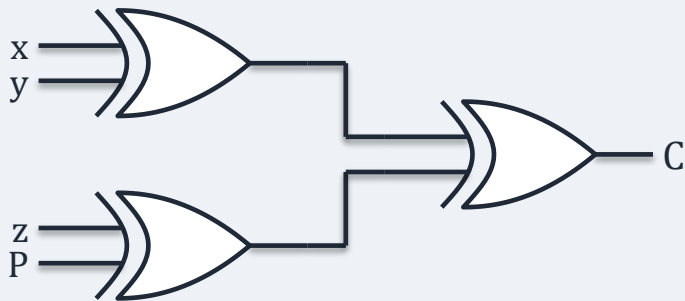
EXAMPLE

Design an even-parity generator for a 3-bit message xyz and the checker at the receiver.

Generator: $P = x \oplus y \oplus z$



Checker: $C = x \oplus y \oplus z \oplus P$



- 1 Even parity: the total number of 1s in $xyzP$ is even
- 2 So $P = 1$ exactly when xyz has an odd count of 1s
- 3 An odd function: $P = x \oplus y \oplus z$
- 4 Message 011: $0 \oplus 1 \oplus 1 = 0 \rightarrow$ send **0110**
- 5 Message 101: $1 \oplus 0 \oplus 1 = 0 \rightarrow$ send **1010**
- 6 Receiver: $C = x \oplus y \oplus z \oplus P$
- 7 $C = 1$ flags an error (an odd number of flips)

Your turn 2 — XOR and parity

§3.8

YOUR TURN

≈ 10 min

- (a) Show algebraically that $x \oplus y' = (x \oplus y)'$.
- (b) Write the ODD-parity bit for a 4-bit message $wxyz$ as an XOR expression. What is it for 1101?
- (c) With odd parity, the receiver gets 1101 1. Is an error detected?

Odd parity: the total number of 1s, including P, is odd.

Your turn 2 — Solution

§3.8

SOLUTION

- 1 (a) Definition: $a \oplus b = ab' + a'b$
- 2 With $b = y'$: $x \oplus y' = x(y')' + x'y'$
- 3 Involution: $= xy + x'y'$
- 4 That is the XNOR: $(x \oplus y)'$ ✓
- 5 (b) Odd parity: total 1s, including P , is odd
- 6 Invert the even bit: $P = (w \oplus x \oplus y \oplus z)'$
- 7 1101: $1 \oplus 1 \oplus 0 \oplus 1 = 1$, so $P = 1' = 0$
- 8 Send **1101 0**
- 9 (c) 1101 1 has four 1s: an even count
- 10 Odd parity needs odd: error **detected**; ask again

Gate-level Verilog with delays

§3.9

Example 1 as a Verilog netlist

```
module ex1_nand (input x, y, z,
                  output F);
    wire xn, yn, zn, t1, t2;
    not #5  (xn, x), (yn, y), (zn, z);
    nand #10 g1 (t1, x,  yn);
    nand #10 g2 (t2, xn, y);
    nand #10 g3 (F,  t1, t2, zn);
endmodule
```

Primitives list the output first, then the inputs. #10 is a delay of 10 time units.

What the simulator shows

Every path: NOT (5) → NAND (10) → NAND (10).

After an input change, F settles within 25 time units.

Before that, F may show the old value or a glitch.

Testbench

Apply all 8 input rows, wait longer than 25 units each, and compare F with $\Sigma m(1,2,3,4,5,7)$.

Concept check

1

Why do designers prefer NAND and NOR gates to AND and OR?

2

In a NAND-NAND circuit, what happens to a single literal term?

3

Which form is natural for NOR-NOR: SOP or POS?

4

Why can't the three-variable odd function be simplified on a map?

Key takeaways

Universal

NAND alone · NOR alone

SOP → NAND

AND-invert, invert-OR

POS → NOR

OR-invert, invert-AND

Bubbles

must meet in pairs

Odd function

$$x \oplus y \oplus z$$

Parity

$$P = x \oplus y \oplus z$$

Push bubbles until they pair up, and every circuit is NAND or NOR.

Homeworks #4 and #5 due at the start of Lecture 9

Next: tabular (Quine–McCluskey) minimization